

5.3

1. Copy the egress log into your **Egress Log** spreadsheet tab. It is space-separated; use your spreadsheet's text-to-columns feature to split it.
2. Sort by date and time. Identify the earliest row involving 185.220.101.47 and record that timestamp. This is your revised start of intrusion.

The earliest activity involving 185.220.101.47 occurred on 2026-08-28 at 01:52:17 UTC. This revises the suspected start of the intrusion to 28 August 2026, several days earlier than the activity initially identified during the Day 3 investigation.

3. Compare that date against the start of your Day 3 timeline. Calculate the difference in days. This is how much of the intrusion your disk evidence could not see.

The revised start of the intrusion was 28 August 2026 at 01:52:17 UTC, while the Day 3 timeline began on 2 September 2026 at 02:12:33 UTC. The disk-based investigation therefore missed approximately five days of earlier attacker activity.

4. Explain in writing why the disk image could not show you this activity. Two reasons apply: one about log rotation, one about where the evidence lives.

The disk image could not show the earlier activity because the Apache logs on MTB-WEB01 had rotated, removing older records before the image was acquired. The perimeter firewall records were also stored outside MTB-WEB01, so that evidence was never present on the server's disk image.

5. Find the row on 30 August where MTB-WEB01 first connects outbound to the attacker. Note what this means: the compromise on 2 September was not the first one.

MTB-WEB01 first connected outbound to the attacker at 02:09:12 UTC on 30 August 2026. The connection went from MTB-WEB01 to 185.220.101.47 on TCP port 8443 and was allowed by the firewall. This indicates that MTB-WEB01 was already compromised and communicating with the attacker before the activity observed on 2 September.

6. Identify every distinct external destination address in the log. There are three. For each, note which internal hosts talked to it.

The three external addresses and associated internal hosts were:

185.220.101.47 — MTB-WEB01
185.220.102.244 — MTB-FILE01
52.168.117.171 — MTB-DVWA01

7. Sort by bytes_out, largest first. Record the two largest rows with their host, timestamp and volume. Convert the byte counts into megabytes.

The two largest outbound transfers were:

MTB-FILE01 transferred 9,917,722 bytes, approximately 9.92 MB, to 185.220.102.244 on 2 September 2026 at 05:12:19 UTC.

MTB-WEB01 transferred 2,884,416 bytes, approximately 2.88 MB, to 185.220.101.47 on 2 September 2026 at 02:38:02 UTC.

8. One of those two large transfers is from a host you have not investigated yet. Note it. You will confirm it in Section 6.

The largest transfer came from MTB-FILE01, a host that had not yet been investigated. Its approximately 9.92 MB outbound transfer to 185.220.102.244 indicates that the investigation must expand beyond MTB-WEB01 to determine whether MTB-FILE01 was also compromised.

5.4 Record: the extended timeline

Date and time (UTC)	Host	Event	Evidence source
2026-08-28 1:52:17	MTB-WEB01	External address 185.220.101.47 made its earliest recorded connection to MTB-WEB01 on TCP port 80.	Perimeter firewall egress log
2026-08-28 1:52:19	MTB-WEB01	External address 185.220.101.47 generated 214 connections to TCP port 80 with no transferred data, indicating scanning or automated probing.	Perimeter firewall egress log
2026-08-28 1:58:44	MTB-WEB01	The firewall denied six connection attempts from 185.220.101.47 to TCP port 443.	Perimeter firewall egress log
2026-08-28 2:03:11	MTB-WEB01	The firewall denied three connection attempts from 185.220.101.47 to TCP port 22.	Perimeter firewall egress log
2026-08-30 2:09:12	MTB-WEB01	MTB-WEB01 connected outbound to 185.220.101.47 on TCP port 8443, confirming compromise and attacker communication before 2 September.	Perimeter firewall egress log
2026-09-02 2:24:48	MTB-WEB01	External address 185.220.101.47 successfully connected to MTB-WEB01 over SSH on TCP port 22.	Perimeter firewall egress log
2026-09-02 2:38:02	MTB-WEB01	MTB-WEB01 transferred 2,884,416 bytes, approximately 2.88 MB, to 185.220.101.47 on TCP port 8443.	Perimeter firewall egress log
2026-09-02 2:57:40	MTB-FILE01	MTB-FILE01 began making regular outbound connections to 185.220.102.244 on TCP port 443.	Perimeter firewall egress log
2026-09-02 5:12:19	MTB-FILE01	MTB-FILE01 transferred 9,917,722 bytes, approximately 9.92 MB, to 185.220.102.244 on TCP port 443.	Perimeter firewall egress log
2026-09-02 9:20:03	MTB-DVWA01	MTB-DVWA01 continued its pre-existing five-minute outbound connection pattern to 52.168.117.171 on TCP port 443.	Perimeter firewall egress log

6.1

Host	185.220.101.47	185.220.102.244	apache-worker	cron entry	svc-backup	Verdict
MTB-WEB01	YES	NO	YES	YES	YES	COMPROMISED
MTB-APP01	NO	NO	NO	NO	NO	CLEAN

Host	185.220.101.47	185.220.102.244	apache-worker	cron entry	svc-backup	Verdict
MTB-FILE01	NO	YES	NO	NO	YES	COMPROMISED
MTB-DC01	NO	NO	NO	NO	NO	CLEAN
MTB-DVWA01	NO	NO	NO	NO	NO	NOT COMPROMISED

6.5

1. Fill in the grid in Section 6.1 for all five hosts, using the logs and telemetry above. Every cell gets an answer.
2. Identify the host that shows an SSH login originating from MTB-WEB01. Record the exact timestamp, the account used, and how long after the MTB-WEB01 root compromise it occurred.

MTB-FILE01 received an SSH login from MTB-WEB01 at 02:52:14 UTC on 2 September 2026. The attacker authenticated using the svc-backup account from 10.50.2.24. This occurred 31 minutes and 9 seconds after the attacker obtained root privileges through pkexec at 02:21:05 UTC, and 27 minutes and 26 seconds—approximately 28 minutes—after the attacker established direct SSH access as root at 02:24:48 UTC.

3. Explain how the attacker was able to authenticate to that second host. Your Day 3 evidence tells you what was sitting on MTB-WEB01 that made this possible.

The attacker could authenticate to MTB-FILE01 because an SSH private key was stored on the already-compromised MTB-WEB01. After gaining root access, the attacker could steal and use that key to make a public-key-authenticated SSH connection to MTB-FILE01 as svc-backup. The MTB-FILE01 authentication log confirms this with “Accepted publickey for svc-backup from 10.50.2.24.”

4. On that second host, identify the beacon. Note three ways it differs from the one on MTB-WEB01: the destination address, the file path, and the persistence mechanism.

The beacon on MTB-FILE01 is /usr/local/sbin/syslog-forward, running as root and connected to 185.220.102.244 on TCP port 443. It differs from the MTB-WEB01 beacon in three ways: it uses 185.220.102.244:443 instead of 185.220.101.47:8443, its file path is /usr/local/sbin/syslog-forward instead of /usr/local/bin/apache-worker, and it is maintained by a systemd service instead of a cron entry. The authentication log supports the systemd persistence finding because it records systemd reloading at 02:56:02 UTC and starting “System Log Forwarder” at 02:56:03 UTC.

5. Add columns to your grid for the new indicators you just found, and re-check every host against them.
6. Look at MTB-DVWA01. It has a persistent external session on a regular five-minute interval, which is exactly the pattern that got MTB-WEB01 flagged. Decide whether it is compromised, and write down the specific evidence that settles it.

MTB-DVWA01 is not compromised. Its regular outbound connections are generated by the approved Meridian Trust monitoring agent, monagent 4.2.1, running from /opt/monagent/bin/monagent. The asset register confirms that the agent checks in every 300 seconds, or five minutes, to the approved endpoint 52.168.117.171:443. Its 31-day process uptime and network activity before the intrusion further confirm that this is expected monitoring traffic rather than an attacker beacon.

7. For the two hosts you conclude are clean, write down exactly what you checked. Not “nothing found” - list the indicators and the log sources.

MTB-APP01: I checked the fleet authentication logs for logins involving svc-backup, connections originating from MTB-WEB01, and other unusual successful access. I also checked the endpoint process and network telemetry for connections to 185.220.101.47 and 185.220.102.244, the apache-worker and syslog-forward processes, the malicious

cron entry, and the System Log Forwarder service. None of these indicators were present. The perimeter firewall egress log also contained no suspicious external traffic from MTB-APP01.

MTB-DC01: I checked the Windows authentication events for suspicious accounts, unusual source addresses, and unauthorized successful logins. I also checked the endpoint process and network telemetry for both attacker addresses, the malicious process names, svc-backup, and the identified persistence mechanisms. None of these indicators were present, and the perimeter firewall egress log contained no suspicious external traffic from MTB-DC01.

- 8. Look at the failed logins on MTB-APP01 at 11:20. Decide whether they are related to this incident and justify your answer using the source address and the timing.

The failed oracle login attempts against MTB-APP01 are not linked to this incident based on the available evidence. They originated from the internal address 10.50.1.99 at 11:20 UTC, rather than from MTB-WEB01 or either identified external attacker address, and they occurred after the SOC alert at 09:47 UTC. All attempts failed, and the endpoint and firewall telemetry show no matching beacon, persistence mechanism, or suspicious external connection on MTB-APP01. The attempts should still be reviewed separately as possible internal scanning or a misconfigured service, but they do not currently indicate that MTB-APP01 was compromised in this attack.

7.1 The Pyramid of Pain

Tier	Indicator type	Pain caused	Example from this incident
6 (top)	TTPs	Tough	A beacon process launched by a persistence mechanism, holding a long-lived outbound session on a fixed interval
5	Tools	Challenging	The specific beacon implementation, whichever filename it is given
4	Network or host artifacts	Annoying	A root-owned process in /usr/local/ with a persistence entry created outside a change window
3	Domain names	Simple	(none observed in this incident)
2	IP addresses	Easy	185.220.101.47 and 185.220.102.244
1 (bottom)	Hash values	Trivial	The SHA-256 of style.php and apache-worker

7.2

- 1. List every indicator from Day 3 and every new one from today. There should be at least sixteen.

185.220.101.47
Outbound TCP port 8443
/var/www/html/uploads/style.php
/usr/local/bin/apache-worker
/etc/cron.d/apache-logrotate
/tmp/.ICE-unix
svc-backup account with UID 0
SSH key comment attacker@blackfin
User agent curl/7.81.0
SHA-256: 2dfc22e2fb93187b7537d9ffec52ecc94fd6416c84b78e68ff7a025e975c20c
SHA-256: 1ebcce99569f4337f9ee3746b7b095a9f54abfe3036ef06014b1a00ea4d15d4b
185.220.102.244

/usr/local/sbin/syslog-forward

Systemd service named **System Log Forwarder**

Accepted public-key SSH login to MTB-FILE01 using svc-backup from MTB-WEB01

Root-owned process in /usr/local/ maintaining a long-lived external connection

Approximately 9.92 MB transferred from MTB-FILE01 to an external address

2. Assign each one a Pyramid tier from the table in Section 7.1.
3. For each indicator at tier 1 or 2, write down how long it would take the attacker to change it and carry on. Be honest - the answer is usually seconds or minutes.

The attacker could replace either IP address within minutes by moving the command-and-control service to another address or proxy. The file hashes could be changed in seconds or minutes by making even a minor change to the files' contents or rebuilding the payload. These indicators are inexpensive for the attacker to replace, which is why hashes and IP addresses occupy the bottom of the Pyramid of Pain.

4. Write three tier 4 to 6 descriptions of this intrusion that do not name a single address, filename or hash. These become the basis of your Day 5 detection rules, so make them specific enough to test.
5. Explain in two or three sentences why an indicator list is still worth producing even though most of it sits at the bottom two tiers.

An indicator list is still valuable because hashes and IP addresses allow security teams to quickly search existing logs, identify affected systems, and block known malicious activity. These indicators are easy for attackers to change, but they provide immediate protection while broader behavioral detections are developed. When combined with higher-tier indicators, they also help connect individual alerts to the larger intrusion.

7.3 Record: tiered indicator list

Tier	Indicator or behaviour description	Source	Attacker cost to change
6	Accepted public-key SSH login to MTB-FILE01 using svc-backup from a compromised host	Section 6.2 MTB-FILE01 authentication log	Days or weeks; obtain different credentials or develop another lateral-movement technique
6	Root-owned process in /usr/local/ maintaining a long-lived external connection	Section 6.3 endpoint telemetry	Days or weeks; redesign the command-and-control behavior and supporting tooling
6	A root-owned process launched by a persistence mechanism maintains a long-lived outbound connection and reconnects at a fixed interval.	Day 3 forensic timeline and Section 6.3 endpoint telemetry	Days or weeks; redesign the beacon and persistence behavior
6	A compromised internet-facing server uses public-key SSH authentication and a privileged service account to move laterally to another internal server.	Section 6.2 MTB-FILE01 authentication log	Days or weeks; obtain another internal access

Tier	Indicator or behaviour description	Source	Attacker cost to change
			path and change the lateral-movement method
4	Outbound TCP port 8443	Day 3 network evidence and Section 5.1 perimeter firewall log	Seconds or minutes; switch to another outbound port
4	/var/www/html/uploads/style.php	Day 3 filesystem timeline and acquisition records	Seconds or minutes; rename or relocate the webshell
4	/usr/local/bin/apache-worker	Day 3 filesystem and process evidence	Seconds or minutes; rename or relocate the beacon
4	/etc/cron.d/apache-logrotate	Day 3 filesystem timeline and shell history	Minutes; create a differently named scheduled task or use another persistence method
4	/tmp/.ICE-unix	Day 3 filesystem timeline and shell history	Seconds; use a different staging directory
4	svc-backup account with UID 0	Day 3 authentication log	Minutes; create another privileged account with a different name
4	SSH key comment attacker@blackfin	Day 3 authorized_keys evidence	Seconds; remove or change the SSH key comment
4	User agent curl/7.81.0	Day 3 Apache access log	Seconds; change the HTTP user-agent string
4	/usr/local/sbin/syslog-forward	Section 6.3 endpoint telemetry	Seconds or minutes; rename or relocate the beacon
4	Systemd service named System Log Forwarder	Section 6.2 authentication log and Section 6.3	Minutes; rename the service or use another

Tier	Indicator or behaviour description	Source	Attacker cost to change
		endpoint telemetry	persistence mechanism
4	Approximately 9.92 MB transferred from MTB-FILE01 to an external address	Section 5.1 perimeter firewall log	Minutes or hours; split, compress, encrypt, or throttle the transfer
4	A new root-owned executable is placed under /usr/local/ outside an approved change window and is configured to start automatically.	Day 3 filesystem timeline and Section 6.3 endpoint telemetry	Hours or days; change both the executable location and automatic-start behavior
2	185.220.101.47	Day 3 network evidence and Section 5.1 perimeter firewall log	Minutes; move the command-and-control service to another IP or proxy
2	185.220.102.244	Section 5.1 perimeter firewall log and Section 6.3 endpoint telemetry	Minutes; move the command-and-control service to another IP or proxy
1	SHA-256: 2dfc22e2fb93187b7537d9ffec52ecc94fd6416c84b78e68ff7a025e975c20c	Day 3 acquisition hash record	Seconds; make a minor change to the file contents
1	SHA-256: 1ebcce99569f4337f9ee3746b7b095a9f54abfe3036ef06014b1a00ea4d15d4b	Day 3 acquisition hash record	Seconds or minutes; modify or rebuild the payload

8. Report

1. Executive Summary

The earliest identified attacker activity occurred on 28 August 2026, approximately five days before the original investigation timeline began. Two servers, MTB-WEB01 and MTB-FILE01, are confirmed compromised with high confidence. Approximately 2.88 MB left the customer web server and 9.92 MB left the internal file server, placing customer portal data and potentially sensitive internal files at risk. Additional forensic analysis is required to determine the exact contents of the second transfer and whether the attacker accessed any other systems.

2. Revised Timeline

The earliest identified attacker activity occurred on 28 August 2026 at 01:52:17 UTC, when the perimeter firewall recorded traffic from 185.220.101.47 to MTB-WEB01. The Day 3 timeline began on 2 September 2026 at 02:12:33 UTC, meaning the attacker's activity began 5 days, 20 minutes, and 16 seconds earlier than the original timeline showed.

The disk image could not reveal this earlier activity because MTB-WEB01's Apache logs had rotated and the longer-retained perimeter firewall records were stored outside the affected server.

The SOC detected the intrusion on 2 September 2026 at 09:47:12 UTC. The confirmed dwell time from the earliest recorded attacker activity to detection was 5 days, 7 hours, 54 minutes, and 55 seconds.

Dwell-time calculation:

28 August 2026 at 01:52:17 UTC to 2 September 2026 at 09:47:12 UTC = 5 days, 7 hours, 54 minutes, and 55 seconds.

3. Confirmed Compromised Hosts

MTB-WEB01 — High confidence

MTB-WEB01 is confirmed compromised. The Apache evidence shows successful path traversal and command execution, followed by the creation and use of the webshell `/var/www/html/uploads/style.php`. The attacker exploited the local privilege-escalation vulnerability to obtain root access, created the UID 0 account `svc-backup`, added an SSH key, and established persistence through `/etc/cron.d/apache-logrotate`. Endpoint and firewall evidence also shows the root-owned beacon `/usr/local/bin/apache-worker` communicating with 185.220.101.47:8443.

MTB-FILE01 — High confidence

MTB-FILE01 is confirmed compromised. Its authentication log records an accepted public-key login for `svc-backup` from MTB-WEB01 at 02:52:14 UTC. The attacker then created a UID 0 `svc-backup` account, reloaded `systemd`, and started the System Log Forwarder service. Endpoint telemetry shows the root-owned process `/usr/local/sbin/syslog-forward` maintaining a connection to 185.220.102.244:443, while the perimeter firewall recorded a large outbound transfer from the host.

4. Hosts Assessed as Clean

MTB-APP01

The authentication logs were checked for `svc-backup`, successful logins originating from MTB-WEB01, and other unusual successful access. Endpoint and firewall telemetry were checked for both attacker IP addresses, the `apache-worker` and `syslog-forward` processes, the malicious cron entry, the System Log Forwarder service, and suspicious outbound connections. None of these indicators were present. The failed oracle login attempts from 10.50.1.99 should be reviewed separately, but they were unsuccessful and are not linked to this intrusion by the available evidence.

MTB-DC01

Windows authentication events were checked for `svc-backup`, unusual source systems, unauthorized successful logins, and activity originating from MTB-WEB01. Endpoint and firewall telemetry were also checked for the attacker addresses, malicious process names, persistence mechanisms, and suspicious external connections. None of the identified indicators were present.

MTB-DVWA01

MTB-DVWA01 showed regular outbound connections to 52.168.117.171:443, but the asset register confirms that this is the approved endpoint for `monagent 4.2.1`. The documented check-in interval is 300 seconds, matching the five-minute pattern in the firewall records. The process had been running for 31 days, and the same activity existed before the intrusion, supporting the conclusion that this was legitimate monitoring traffic. These clean assessments would detect the currently known accounts, processes, persistence mechanisms, network destinations, and lateral-movement pattern. They would not necessarily detect an attacker using completely different tools, credentials, infrastructure, or behavior.

5. Lateral Movement

The attacker initially reached MTB-WEB01 by exploiting the path traversal and remote-command-execution weakness in Apache 2.4.49, enabled by the server's exposed CGI configuration and permissive access directive. The attacker uploaded a webshell and executed commands as the web-server account before exploiting the PwnKit vulnerability to obtain root privileges.

After obtaining root access, the attacker accessed an SSH private key stored on MTB-WEB01. That key was used for public-key authentication from MTB-WEB01 to MTB-FILE01 as svc-backup at 02:52:14 UTC. On MTB-FILE01, the attacker created another UID 0 svc-backup account and installed a root-owned beacon maintained by a systemd service.

6. Data at Risk

MTB-WEB01 transfer

At 02:38:02 UTC on 2 September 2026, MTB-WEB01 transferred 2,884,416 bytes, approximately 2.88 MB, to 185.220.101.47:8443. The Day 3 filesystem timeline shows that data from the customer portal was archived shortly beforehand. This supports the assessment that customer portal data likely left the bank, although the exact records contained in the transfer have not been confirmed.

MTB-FILE01 transfer

At 05:12:19 UTC on 2 September 2026, MTB-FILE01 transferred 9,917,722 bytes, approximately 9.92 MB, to 185.220.102.244:443. Because MTB-FILE01 is an internal file server, the transfer may contain internal bank documents or shared business files. The firewall log proves the outbound volume but does not reveal the transfer's contents, so the specific files remain unknown.

7. Intelligence Gaps

Exact time of initial compromise: The firewall establishes attacker activity beginning on 28 August, but it does not prove exactly when the attacker first achieved command execution. Older Apache, proxy, web-application firewall, SIEM, or backup logs would be needed to identify the first successful exploit.

Contents of the MTB-FILE01 transfer: The firewall records the transfer volume but not its contents. A forensic image of MTB-FILE01, file-access auditing, shell history, process memory, deleted-file recovery, and any available proxy or packet-capture evidence would be required to identify the files transferred.

Full extent of customer-data exposure: The evidence indicates that customer portal data was staged and likely transferred, but the specific records are not known. The recovered archive, web-server backups, application audit logs, and database-access records would help determine which customers and fields were affected.

Possible additional lateral movement: No current evidence confirms compromise of the remaining hosts, but indicator-based checks cannot rule out different attacker tooling. Broader authentication, EDR, DNS, firewall, and network-flow records across other network segments should be reviewed for behavioral signs of lateral movement.

Origin and use of the SSH private key: The evidence supports the conclusion that a private key on MTB-WEB01 enabled access to MTB-FILE01, but its original owner and every system where it was accepted remain unknown. The key inventory, SSH configuration, authorized-key files, and authentication logs from all systems that trusted the key would close this gap.

9.3

Tactic	Technique ID	Technique name	Your evidence
Reconnaissance	T1595.002	Vulnerability Scanning	Egress log, 28 Aug 01:52, 214 sessions to port 80 carrying zero bytes

Tactic	Technique ID	Technique name	Your evidence
Initial Access	T1190	Exploit Public-Facing Application	MTB-WEB01, 2 Sep 02:12:33 UTC, Apache access log: a path-traversal request against Apache 2.4.49 successfully returned /etc/passwd.
Persistence	T1505.003	Web Shell	MTB-WEB01, 2 Sep 02:15:22 UTC, filesystem timeline: /var/www/html/uploads/style.php was created; the Apache access log shows it being used at 02:17:40.
Privilege Escalation	T1068	Exploitation for Privilege Escalation	MTB-WEB01, 2 Sep 02:21:05 UTC, authentication log: www-data used pkexec to execute /bin/sh as root.
Persistence	T1136.001	Local Account	MTB-WEB01, 2 Sep 02:23:11 UTC, authentication log: svc-backup was created with UID 0; MTB-FILE01 recorded the same account creation at 02:54:39.
Persistence	T1098.004	SSH Authorized Keys	MTB-WEB01, 2 Sep 02:24:48 UTC, filesystem timeline and authentication evidence: an attacker-controlled key was added to /root/.ssh/authorized_keys and used for root SSH access.
Persistence	T1053.003	Cron	MTB-WEB01, 2 Sep 02:26:30 UTC, filesystem timeline: /etc/cron.d/apache-logrotate was created to launch the beacon every five minutes.
Persistence	T1543.002	Systemd Service	MTB-FILE01, 2 Sep 02:56:03 UTC, authentication/system log: systemd started the System Log Forwarder service.
Credential Access	T1552.004	Private Keys	MTB-WEB01 and MTB-FILE01, 2 Sep 02:52:14 UTC, MTB-FILE01 authentication log: an accepted public-key login originated from compromised MTB-WEB01, supporting use of the SSH private key stored there.
Lateral Movement	T1021.004	SSH	MTB-FILE01, 2 Sep 02:52:14 UTC, authentication log: svc-backup logged in through SSH using public-key authentication from MTB-WEB01.
Collection	T1560.001	Archive via Utility	MTB-WEB01, 2 Sep 02:34:55 UTC, shell history and filesystem timeline: tar was used to archive customer portal data in a hidden temporary directory.
Command and Control	T1071.001	Web Protocols	MTB-WEB01, 2 Sep 02:31:12 UTC, perimeter log and endpoint telemetry: the root-owned beacon connected to

Tactic	Technique ID	Technique name	Your evidence
			185.220.101.47:8443; MTB-FILE01 began communicating with 185.220.102.244:443 at 02:57:40.
Exfiltration	T1041	Exfiltration Over C2 Channel	MTB-WEB01, 2 Sep 02:38:02 UTC, perimeter log: approximately 2.88 MB was sent over the C2 connection; MTB-FILE01 sent approximately 9.92 MB over its C2 connection at 05:12:19.
Stealth	T1070.003	Clear Command History	MTB-WEB01, 2 Sep 02:41:22 UTC, filesystem timeline: the mtbadm shell-history file was reduced to zero bytes after the attacker attempted to clear command history.
Stealth	T1027	Obfuscated Files or Information	MTB-WEB01, 2 Sep 02:15:22 UTC, recovered file and CyberChef analysis: the Base64-encoded content inside style.php decoded to the lab's simulated webshell marker.
Collection	T1074.001	Local Data Staging	MTB-WEB01, 2 Sep 02:34:55 UTC, filesystem timeline and root shell history: customer portal data was staged locally in /tmp/.ICE-unix/x before the large outbound transfer.
Stealth	T1036	Masquerading	MTB-WEB01, 2 Sep 02:25:57 UTC, filesystem timeline and endpoint telemetry: the beacon and cron entry were given names resembling legitimate Apache components; MTB-FILE01 used a process and systemd service named to resemble log-forwarding software at 02:56:03.

9.4

1. Complete the evidence column for all fifteen rows. Each entry needs a host, a timestamp and a source.
2. Two techniques in the table appear on both compromised hosts but with different implementations. Identify them and note both implementations.

T1071.001, Web Protocols, appeared on both compromised hosts through differently named beacon processes, different destination addresses, and different ports. T1041, Exfiltration Over C2 Channel, also appeared on both hosts, with MTB-WEB01 transferring approximately 2.88 MB through its C2 channel and MTB-FILE01 transferring approximately 9.92 MB through a separate C2 channel. This demonstrates that the attacker changed the visible infrastructure and tooling while preserving the same underlying behaviors.

3. One technique in the table is evidenced by something you decoded rather than something you observed running. Identify it.

T1027, Obfuscated Files or Information, is the technique demonstrated by the evidence decoded in CyberChef. The recovered style.php file, created on MTB-WEB01 at 02:15:22 UTC, contained a Base64-encoded string that decoded to

“MTB-LAB-ARTIFACT: simulated webshell payload, no executable content.” This evidence came from examining and decoding the recovered file rather than observing the encoded payload running.

4. Add at least two techniques that are not in the table but that you have evidence for. Appendix A.1 lists the full set used in this incident if you want to check your work afterwards - try it yourself first.

Two additional techniques were identified. T1074.001, Local Data Staging, occurred when customer portal data was archived in a hidden temporary directory before exfiltration. T1036, Masquerading, occurred when the attacker gave malicious processes and persistence entries names that resembled legitimate Apache and system logging components.

5. For each of the fifteen ATT&CK tactics, note whether this attacker used it. The ones they did not use are as informative as the ones they did.

Reconnaissance - YES. The attacker performed automated vulnerability scanning against MTB-WEB01.

Resource Development - NO CONFIRMED EVIDENCE. The attacker used external infrastructure, but the evidence does not show how that infrastructure was obtained or prepared.

Initial Access - YES. The attacker exploited the public-facing Apache server.

Execution - YES. The attacker executed commands through the webshell and a root shell.

Persistence - YES. Persistence included a webshell, a UID 0 account, an authorized SSH key, a cron entry, and a systemd service.

Privilege Escalation - YES. The attacker exploited pkexec to move from www-data to root.

Stealth - YES. The attacker cleared command history, encoded the webshell content, and disguised malicious components with legitimate-looking names.

Defense Impairment - NO CONFIRMED EVIDENCE. There is no evidence that the attacker disabled security software, logging pipelines, or other defensive controls.

Credential Access - YES. The attacker obtained and used an SSH private key stored on MTB-WEB01.

Discovery - YES. The root shell history shows commands used to identify the account, operating system, kernel, and available utilities.

Lateral Movement - YES. The attacker used SSH public-key authentication to move from MTB-WEB01 to MTB-FILE01.

Collection - YES. Customer portal data was gathered, archived, and staged before transfer.

Command and Control - YES. Root-owned beacon processes maintained external command-and-control connections from both compromised hosts.

Exfiltration - YES. Large outbound transfers occurred from MTB-WEB01 and MTB-FILE01 through their command-and-control channels.

Impact - NO CONFIRMED EVIDENCE. There is no evidence that the attacker destroyed, encrypted, altered, or disrupted bank systems or data.

10.2

Field	Required?	What it holds
title	Mandatory	A short description of what the rule detects. Keep it under about 50 characters.
logsource	Mandatory	Which logs to run against. Built from category, product and service.
detection	Mandatory	What to match. Contains one or more named search identifiers plus a condition.
condition	Mandatory	Inside detection. How the named searches combine, for example 'selection and not filter'.
id	Optional	A UUID so the rule can be referenced. Generate one or leave it out for this lab.
status	Optional	One of stable, test, experimental, deprecated, unsupported.
description	Optional	Fuller explanation, including why the rule exists.
references	Optional	Links justifying the rule, for example the ATT&CK technique page.
author	Optional	Who wrote it.
date	Optional	ISO 8601 format, YYYY-MM-DD. Not any other format.
tags	Optional	Namespaced and lower case, for example attack.t1505.003.
falsepositives	Optional	What legitimate activity could also match. Never leave this empty in practice.
level	Optional	One of informational, low, medium, high, critical.

10.4

- **It sits high on the Pyramid.** It does not name an account, an address or a file. Any attacker creating a UID 0 account on any host trips it. To evade it they must stop using that technique.
- **It fires on the attacker's own noise.** The operating system generates that warning by itself. The attacker cannot suppress it without also failing to create the account.
- **Its false positive case is named and realistic.** A break-glass account is a real thing that would match. Saying so tells the SOC how to triage a hit rather than leaving them guessing.
- **It would have fired twice in this incident,** at 02:23:11 on MTB-WEB01 and again at 02:54:39 on MTB-FILE01, the second one 53 minutes before any alert actually reached a human.

10.5

Rule	What it must detect	Evidence it must catch
mtb_detect_01	A persistence entry, cron or systemd, that executes a binary from /usr/local/	/etc/cron.d/apache-logrotate on MTB-WEB01 and the System Log Forwarder unit on MTB-FILE01
mtb_detect_02	A file with a script extension written into a directory that the web server serves and that accepts uploads	style.php appearing in /var/www/html/uploads/ at 02:15:22
mtb_detect_03	A web request whose path contains encoded directory traversal against a CGI path	The /cgi-bin/.%2e/.%2e/ requests on 30 August and 2 September
mtb_detect_04	An SSH login accepted for a service account originating from a server-segment host rather than a workstation	svc-backup accepted from 10.50.2.24 on MTB-FILE01 at 02:52:14

10.7

Rule file	What it detects	ATT&CK tag	Level	Pyramid tier
mtb_detect_01	Cron or systemd launching a binary from /usr/local/	attack.t1053.003, attack.t1543.002	High	Tier 6
mtb_detect_02	A server-side script written into a web upload directory	attack.t1505.003	High	Tier 4
mtb_detect_03	Encoded directory traversal targeting a CGI path	attack.t1190	High	Tier 6
mtb_detect_04	Accepted SSH public-key login for a service account originating from the server segment	attack.t1021.004	Medium	Tier 6

Detection Rule Explanations

Then paste all four entries directly underneath it:

mtb_detect_01.yml: This rule targets Pyramid Tier 6 because it detects persistence behavior rather than a specific filename or IP address. To evade it, the attacker would need to change the executable location, use a different persistence mechanism, or avoid creating a persistent process.

mtb_detect_02.yml: This rule targets Pyramid Tier 4 because it detects a suspicious host artifact based on its location and script extension rather than one exact filename. To evade it, the attacker would need to use a non-script extension, place the file outside the monitored upload directory, or use a fileless execution method.

mtb_detect_03.yml: This rule targets Pyramid Tier 6 because it detects an encoded directory-traversal technique rather than one attacker address. To evade it, the attacker would need to use a different exploit, avoid the CGI path, or substantially change the traversal encoding.

mtb_detect_04.yml: This rule targets Pyramid Tier 6 because it detects a lateral-movement pattern involving service-account authentication between servers. To evade it, the attacker would need to use a differently named account, originate outside the monitored server segment, or use another lateral-movement protocol.

11.2

Containment Plan

1. Preserve and collect volatile evidence from MTB-WEB01 and MTB-FILE01 while both remain powered on. Capture memory, running processes, active network connections, logged-in users, open files, system time, and current EDR data. This preserves evidence that would be destroyed if either system were shut down, although the collection tools will make small changes to memory and system metadata.
2. After volatile collection, isolate both compromised hosts through EDR network containment and firewall ACLs while leaving them powered on. This stops command-and-control traffic, exfiltration, and lateral movement. Isolation will terminate active network connections and may alert the attacker, which is why those connections must be documented first.
3. Block 185.220.101.47, 185.220.102.244, and unauthorized outbound TCP port 8443 traffic at the perimeter. This stops communication with the currently known attacker infrastructure but does not destroy host evidence. Because the attacker can change addresses and ports quickly, these blocks must supplement rather than replace behavioral monitoring.

4. Disable the svc-backup account across the environment and revoke the attacker-controlled SSH key. This prevents continued privileged access and further SSH lateral movement. Authentication logs and copies of the relevant account and key files must be preserved before making these changes.
5. Temporarily restrict server-to-server SSH so only approved administrative workstations and jump hosts can initiate connections. This blocks the lateral-movement path used from MTB-WEB01 to MTB-FILE01. The change may interrupt legitimate automated tasks, so system owners must identify approved exceptions.
6. Restrict outbound connections from the server segment to documented business destinations. This stops unknown beaconing and additional data transfers while the investigation continues. Required services may be disrupted until legitimate destinations are reviewed and allowlisted.
7. Preserve forensic disk images and centralized copies of authentication, Apache, firewall, systemd, cron, EDR, and network-flow logs from both hosts. This does not directly stop the attacker, but it prevents evidence from being lost through log rotation, cleanup, or rebuilding.

Containment Decision

MTB-WEB01 and MTB-FILE01 should remain powered on long enough to collect volatile evidence and should then be isolated at the network. Immediately powering them off would destroy memory contents, running-process information, active connections, and logged-in session data. After volatile evidence and forensic disk images are secured, the compromised systems can be shut down and rebuilt from trusted sources.

Containment on Hosts Not Confirmed Compromised

The compromised SSH private key must be identified and revoked on every system that trusted it, not only MTB-WEB01 and MTB-FILE01. All remaining hosts must be searched for the attacker-controlled key, svc-backup or other UID 0 accounts, suspicious files under /usr/local/, unknown cron entries and systemd services, connections to the identified attacker infrastructure, and unusual server-to-server SSH logins. Relevant credentials must be rotated, centralized logs preserved, and heightened monitoring enabled because the attacker may have used different filenames, addresses, or persistence mechanisms elsewhere.

11.3

Access path	Host(s)	How you remove it	How you verify removal
Apache path traversal and remote code execution	MTB-WEB01	Rebuild from a trusted pre-intrusion image, upgrade Apache to 2.4.51 or later, disable CGI if unnecessary, set the filesystem root to Require all denied, and grant access only to required directories.	Confirm the installed Apache version, review the active configuration, run a vulnerability scan, and test that encoded traversal requests are denied.
Webshell at /var/www/html/uploads/style.php	MTB-WEB01	Preserve a forensic copy and then remove it through the trusted rebuild. Search the entire web root for additional unauthorized scripts.	Confirm the file and its hash are absent, compare the web root with a known-good version, and run mtb_detect_02.
PwnKit privilege-escalation vulnerability	MTB-WEB01	Upgrade policykit/pkexec to the current vendor-supported	Confirm the installed package version and run an

Access path	Host(s)	How you remove it	How you verify removal
		package containing the CVE-2021-4034 fix.	authenticated vulnerability scan.
UID 0 svc-backup account	MTB-WEB01 and MTB-FILE01	Preserve the account records, terminate active sessions, remove the account and home directory, and invalidate its password and keys across the environment.	Review /etc/passwd, /etc/shadow, and /etc/group; confirm root is the only authorized UID 0 account; and search centralized authentication logs for further use.
Attacker SSH key and compromised private key	MTB-WEB01, MTB-FILE01, and every host that trusted the key	Remove the attacker's public key, revoke and replace the compromised key pair, and remove the old key from every authorized_keys file where it was trusted.	Search all hosts for the old key and its fingerprint, verify the replacement key inventory, and confirm authentication using the revoked key fails.
apache-worker beacon and cron persistence	MTB-WEB01	Preserve copies, terminate the process, remove /usr/local/bin/apache-worker and /etc/cron.d/apache-logrotate, and rebuild the host from a trusted image.	Confirm the process, file, and cron entry are absent; check for connections to the C2 infrastructure; and run mtb_detect_01.
syslog-forward beacon and System Log Forwarder service	MTB-FILE01	Preserve copies, stop and disable the service, remove its systemd unit and /usr/local/sbin/syslog-forward, reload systemd, and rebuild from a trusted image.	Confirm the service, process, unit, and file are absent; review outbound connections; and run mtb_detect_01.
Staged data and other attacker-created files	MTB-WEB01 and MTB-FILE01	Preserve forensic copies and remove /tmp/.ICE-unix and any other unauthorized staging files during the trusted rebuild.	Search both filesystems for known hashes, hidden staging directories, recently created archives, and other files created during the incident window.

1. Identify every credential and key that must be treated as compromised, and state the blast radius of each. For the SSH private key, that means every host that trusted it, not just the two you found it used on.

The svc-backup account, its password, and any associated keys must be treated as compromised on every host where the account exists or was accepted. The SSH private key harvested from MTB-WEB01 must be revoked and replaced everywhere its corresponding public key was trusted, not only on the two confirmed compromised hosts. Any administrative credentials, application secrets, API tokens, database credentials, or additional private keys accessible to root on MTB-WEB01 or MTB-FILE01 must also be inventoried and rotated because the attacker had complete control of both systems.

2. State how you would confirm eradication is complete. Your Section 10 detection rules are part of the answer.

Both compromised hosts should be rebuilt from trusted images rather than relying only on individual file removal. Eradication is complete only after authenticated vulnerability scans confirm the Apache and PwnKit vulnerabilities are fixed, the Apache configuration is corrected, and searches confirm that no unauthorized UID 0 accounts, SSH keys,

cron entries, systemd services, processes, or attacker files remain. The four Section 10 detection rules must then be run against both rebuilt hosts and retained historical logs, while firewall and endpoint telemetry are checked for renewed command-and-control traffic, lateral movement, or large outbound transfers.

11.4

- 1. State the order in which the two hosts return to service, and why that order.

MTB-FILE01 should return to service first because it is internal and can be tested in a restricted network segment before normal access is restored. MTB-WEB01 should return last because it is internet-facing and was the initial entry point, making it the highest-risk system to expose. Both hosts should initially operate with restricted network access while security monitoring confirms that no malicious activity returns.

- 2. State which backup generation is safe, based on your revised start of intrusion, and how you would verify it before restoring.

Use a backup created before 28 August 2026 at 01:52:17 UTC, preferably the most recent verified backup from 27 August or earlier. Any backup created after that point may contain attacker artifacts. Before restoration, mount the backup offline, verify its stored hash, scan it for the known malicious files and hashes, and inspect its accounts, SSH keys, cron entries jobs, systemd services, web directories, installed packages, and Apache configuration.

- 3. State what must be true before MTB-WEB01 is exposed to the internet again. Both vulnerabilities from your Day 1-2 assessment are part of this, and so is the Apache configuration.

Before MTB-WEB01 is exposed to the internet, Apache must be upgraded to version 2.4.51 or later and policykit/pkexec must be upgraded to a vendor-supported version that fixes CVE-2021-4034. Unnecessary CGI functionality must be disabled, the filesystem root must use Require all denied, and access must be granted only to specifically required web directories. The host must also be rebuilt from a trusted source, all compromised credentials and keys must be rotated, the web root must match a known-good version, the Section 10 detection rules must be active, and vulnerability testing must confirm that encoded traversal and privilege escalation no longer work.

- 4. Define a heightened monitoring period: how long, watching for what, and using which of your four detection rules.

Meridian Trust should apply heightened 24-hour SOC monitoring for at least 30 days after recovery. Analysts should watch for new files in web upload directories, cron or systemd launching binaries from /usr/local/, service-account SSH logins between servers, encoded traversal requests, unauthorized UID 0 accounts, new SSH keys, long-lived external connections, and unusually large outbound transfers. All four Section 10 detection rules should remain active, and any matching activity should trigger immediate investigation and possible host isolation.

12.1

Moment	Timestamp	How long after first activity
First attacker activity	2026-08-28 01:52:17 UTC	0 days
First successful exploitation	2026-08-30 02:07:55 UTC	2 days, 15 minutes, 38 seconds
Second host compromised	2026-09-02 02:52:14 UTC	5 days, 59 minutes, 57 seconds
Largest data transfer	2026-09-02 05:12:19 UTC	5 days, 3 hours, 20 minutes, 2 seconds
Alert reached a human	2026-09-02 09:47:12 UTC	5 days, 7 hours, 54 minutes, 55 seconds

- 1. For each of your four detection rules from Section 10, state when it would have fired. Then state how much of the intrusion would have been prevented if the SOC had acted on that first firing.

mtb_detect_01 would have fired at 02:30:01 UTC on 2 September, when cron first executed the beacon from /usr/local/. This was 7 hours, 17 minutes, and 11 seconds before the human alert. Immediate containment could have prevented the large transfers from both hosts and the subsequent compromise of MTB-FILE01, although the attacker had already obtained root access and established several persistence mechanisms.

mtb_detect_02 would have fired at 02:15:22 UTC on 2 September, when style.php appeared in the web upload directory. This was 7 hours, 31 minutes, and 50 seconds before the human alert. Immediate containment could have prevented privilege escalation, persistent SSH access, lateral movement, and both large outbound transfers.

mtb_detect_03 would have first fired at 02:07:55 UTC on 30 August, when the encoded traversal request targeted the CGI path. This was 3 days, 7 hours, 39 minutes, and 17 seconds before the human alert. Immediate response could have contained MTB-WEB01 before the attacker established continuing access, compromised MTB-FILE01, or transferred the identified data.

mtb_detect_04 would have fired at 02:52:14 UTC on 2 September, when svc-backup logged in to MTB-FILE01 from MTB-WEB01. This was 6 hours, 54 minutes, and 58 seconds before the human alert. Immediate containment could have prevented the UID 0 account creation, systemd persistence, beacon deployment, and 9.92 MB transfer from MTB-FILE01, but the earlier 2.88 MB transfer from MTB-WEB01 had already occurred.

2. Identify which single rule would have made the biggest difference, and say why.

The rule that would have made the biggest difference is mtb_detect_03, which detects encoded directory traversal against a CGI path. It would have fired during the first successful exploitation on 30 August, more than three days before the actual alert. Acting on that detection could have contained the initial web server before the attacker established persistence, returned on 2 September, moved to MTB-FILE01, or conducted the two identified large transfers.

3. Note honestly what none of your rules would have caught. Every detection set has gaps, and a review that claims otherwise is not credible.

None of the four rules would have detected the attacker's reconnaissance on 28 August before an exploit request occurred. They also do not directly detect PwnKit privilege escalation, creation of an unauthorized UID 0 account, modification of SSH authorized keys, command-history clearing, local data staging, or every possible form of command-and-control and exfiltration. An attacker using a different initial-access method, a non-service account, a fileless payload, or persistence outside cron and systemd could therefore avoid this detection set.

12.2

- **Why was a host with a publicly known, actively exploited vulnerability still exposed to the internet in September,** when the assessment on 8 September flagged it and it had been in the KEV catalogue since 2021? Your answer must address the patching process, not the attacker.

MTB-WEB01 remained exposed because the vulnerability-management process did not reliably connect threat intelligence, asset criticality, remediation ownership, deadlines, and verification. A publicly exposed system with an actively exploited vulnerability should have received an emergency remediation deadline, a named owner, and escalation when the deadline was missed. The process also failed to verify that the Apache upgrade and configuration changes had actually been completed before the host remained internet-facing.

- **Why did the assessment run on 8 September, after the first successful exploitation on 30 August?** What scan cadence would have caught this in time, and what would that cost?

The normal assessment occurred too late because the scanning schedule was not frequent enough for an internet-facing banking system. Internet-facing systems should receive at least weekly authenticated vulnerability scans, daily external-exposure monitoring, and an additional scan whenever a high-risk vulnerability is added to KEV or a significant system change occurs. This would require additional scanner capacity, analyst time to validate findings, and more frequent maintenance windows, but those costs are significantly lower than responding to a multi-host compromise and potential data breach.

- **Why was an unattended SSH private key trusted by other hosts sitting on an internet-facing server in the DMZ?** What should the trust relationship between the DMZ and the server segment look like?

The environment allowed a reusable SSH private key on an internet-facing server to be trusted by internal systems, creating a direct lateral-movement path from the DMZ into the server segment. DMZ hosts should not initiate SSH connections directly to internal servers unless there is a documented business requirement enforced through a restricted firewall rule and managed jump host. Keys should be unique, stored in a managed secrets vault, rotated regularly, limited to specific accounts and commands, and never reused broadly across hosts.

- **Why did seven and a half hours of five-minute beaconing to a non-allowlisted destination not alert anyone sooner?** The egress log recorded every one of those connections. The data existed. What was missing?

The necessary firewall records existed, but Meridian Trust lacked detection logic that converted repeated connections to a non-allowlisted destination into an actionable alert. The monitoring process was collecting data without correlating destination reputation, connection regularity, process ownership, and host criticality. A behavioral rule for repeated outbound connections, supported by allowlist comparison and 24-hour SOC alerting, could have detected the beacon near its first occurrence instead of more than seven hours later.

12.3 Report

MTB Post-Incident Review Case MTB-IR-2026-0902

Executive Summary

The investigation confirmed that an attacker operated in Meridian Trust Bank's environment for more than five days before detection and compromised two servers. Approximately 2.88 MB left the customer web server and 9.92 MB left the internal file server, although the exact contents of the second transfer remain unknown. The incident resulted from weaknesses in patch management, network trust, credential handling, and behavioral monitoring. Meridian Trust will rebuild the affected systems, rotate exposed credentials, restrict connections from the DMZ, increase scanning frequency, and deploy new detections before restoring normal service.

What Happened

The earliest identified attacker activity occurred on 28 August 2026 at 01:52:17 UTC, when an external system began probing MTB-WEB01. Automated scanning generated 214 connections to the web server two seconds later. On 30 August, additional inbound activity was followed by an outbound connection from MTB-WEB01 to the attacker, confirming that the server had already been compromised.

The attacker returned on 2 September and exploited the public-facing Apache service. A webshell was placed in the server's upload directory and used to execute commands through ordinary web requests. The attacker then elevated from the restricted web-server account to root, created a second UID 0 account, added an SSH key, and installed a beacon that cron launched every five minutes.

Customer portal data was archived in a hidden temporary directory and approximately 2.88 MB was transferred through the command-and-control connection. The attacker then used an SSH private key stored on MTB-WEB01 to access MTB-FILE01 as a privileged service account. On MTB-FILE01, the attacker created another UID 0 account and installed a differently named beacon maintained by a systemd service. At 05:12:19 UTC, approximately 9.92 MB left the internal file server.

The SOC alert reached an analyst at 09:47:12 UTC on 2 September. MTB-WEB01 was isolated, volatile evidence was collected before the system was powered down, and a forensic image was acquired and verified. The investigation was expanded to the remaining server fleet, confirming MTB-FILE01 as the second compromised host while finding no evidence that MTB-APP01, MTB-DC01, or MTB-DVWA01 participated in the intrusion. Both compromised hosts must be rebuilt from trusted pre-intrusion sources before returning to service.

ATT&CK Mapping

Reconnaissance | T1595.002 | Vulnerability Scanning | MTB-WEB01, 28 Aug 01:52:19 UTC: the perimeter log recorded 214 connections to TCP port 80 with no transferred data.

Initial Access | T1190 | Exploit Public-Facing Application | MTB-WEB01, 2 Sep 02:12:33 UTC: an Apache traversal request successfully returned /etc/passwd.

Persistence | T1505.003 | Web Shell | MTB-WEB01, 2 Sep 02:15:22 UTC: /var/www/html/uploads/style.php was created and subsequently used for command execution.

Privilege Escalation | T1068 | Exploitation for Privilege Escalation | MTB-WEB01, 2 Sep 02:21:05 UTC: www-data used pkeyexec to execute a root shell.

Persistence | T1136.001 | Local Account | MTB-WEB01, 2 Sep 02:23:11 UTC, and MTB-FILE01, 02:54:39 UTC: svc-backup was created with UID 0.

Persistence | T1098.004 | SSH Authorized Keys | MTB-WEB01, 2 Sep 02:24:48 UTC: an attacker-controlled key was placed in root's authorized-keys file.

Persistence | T1053.003 | Cron | MTB-WEB01, 2 Sep 02:26:30 UTC: a cron entry was created to execute the beacon every five minutes.

Persistence | T1543.002 | Systemd Service | MTB-FILE01, 2 Sep 02:56:03 UTC: systemd started the System Log Forwarder service.

Credential Access | T1552.004 | Private Keys | MTB-FILE01, 2 Sep 02:52:14 UTC: a public-key login from compromised MTB-WEB01 supported the use of the SSH private key stored there.

Lateral Movement | T1021.004 | SSH | MTB-FILE01, 2 Sep 02:52:14 UTC: svc-backup logged in through SSH from MTB-WEB01.

Collection | T1560.001 | Archive via Utility | MTB-WEB01, 2 Sep 02:34:55 UTC: tar was used to archive customer portal data.

Command and Control | T1071.001 | Web Protocols | MTB-WEB01 communicated with 185.220.101.47:8443, while MTB-FILE01 communicated with 185.220.102.244:443.

Exfiltration | T1041 | Exfiltration Over C2 Channel | MTB-WEB01 transferred approximately 2.88 MB at 02:38:02 UTC, and MTB-FILE01 transferred approximately 9.92 MB at 05:12:19 UTC.

Stealth | T1070.003 | Clear Command History | MTB-WEB01, 2 Sep 02:41:22 UTC: a shell-history file was reduced to zero bytes.

Stealth | T1027 | Obfuscated Files or Information | MTB-WEB01, 2 Sep 02:15:22 UTC: the recovered webshell contained a Base64-encoded payload marker decoded in CyberChef.

Collection | T1074.001 | Local Data Staging | MTB-WEB01, 2 Sep 02:34:55 UTC: archived customer data was staged in a hidden temporary directory before transfer.

Stealth | T1036 | Masquerading | The attacker used names resembling legitimate Apache and system-logging components for malicious files and services on both compromised hosts.

Detection Gap

Moment | Timestamp | Time after first activity

First attacker activity | 2026-08-28 01:52:17 UTC | 0 days

First successful exploitation | 2026-08-30 02:07:55 UTC | 2 days, 15 minutes, 38 seconds

Second host compromised | 2026-09-02 02:52:14 UTC | 5 days, 59 minutes, 57 seconds

Largest data transfer | 2026-09-02 05:12:19 UTC | 5 days, 3 hours, 20 minutes, 2 seconds

Alert reached a human | 2026-09-02 09:47:12 UTC | 5 days, 7 hours, 54 minutes, 55 seconds

The total dwell time was 5 days, 7 hours, 54 minutes, and 55 seconds.

mtb_detect_01 would have fired at 02:30:01 UTC on 2 September when cron first executed a beacon from /usr/local/.

Immediate response could have prevented the compromise of MTB-FILE01 and both large transfers, although the

attacker already had root access on MTB-WEB01.

mtb_detect_02 would have fired at 02:15:22 UTC on 2 September when the webshell appeared in the upload directory. Immediate response could have prevented privilege escalation, persistent access, lateral movement, and both large transfers.

mtb_detect_03 would have fired at 02:07:55 UTC on 30 August when the encoded traversal request targeted the CGI path. This rule would have made the greatest difference because it could have exposed the compromise more than three days before the actual alert and before the attacker established the later persistence and lateral-movement chain.

mtb_detect_04 would have fired at 02:52:14 UTC on 2 September when the service account logged in to MTB-FILE01 from MTB-WEB01. Immediate action could have prevented persistence and the 9.92 MB transfer from MTB-FILE01, but the earlier transfer from MTB-WEB01 had already occurred.

None of these rules would have detected the reconnaissance before an exploit request occurred. They also do not directly detect the pkexec privilege escalation, every unauthorized UID 0 account, SSH-key modification, command-history clearing, all forms of data staging, or an attacker using entirely different access and persistence techniques.

New Detections

Rule | Detection | ATT&CK tags | Level | Pyramid tier

mtb_detect_01.yml | Cron or systemd launching a binary from /usr/local/ | attack.t1053.003, attack.t1543.002 | High | Tier 6

mtb_detect_02.yml | A server-side script written into a web upload directory | attack.t1505.003 | High | Tier 4

mtb_detect_03.yml | Encoded traversal against a CGI path | attack.t1190 | High | Tier 6

mtb_detect_04.yml | Service-account SSH authentication originating from the server segment | attack.t1021.004 | Medium | Tier 6

The complete rules are provided in the four accompanying .yml files.

mtb_detect_01 detects persistent execution behavior. An attacker would need to change the executable location, persistence mechanism, or method of maintaining access to evade it.

mtb_detect_02 detects a suspicious script based on its extension and location. An attacker would need to use another directory, a non-script extension, or fileless execution to evade it.

mtb_detect_03 detects the exploitation pattern rather than an attacker address. An attacker would need to use a different exploit, avoid the CGI path, or substantially alter the traversal encoding.

mtb_detect_04 detects service-account lateral movement between servers. An attacker would need to change the account naming pattern, connection source, or lateral-movement protocol.

Containment, Eradication and Recovery

Containment

Responders must first capture memory, running processes, network connections, logged-in users, and open files from both compromised hosts. After volatile collection, MTB-WEB01 and MTB-FILE01 must be isolated through EDR containment and firewall ACLs while remaining powered on for evidence preservation. Known attacker addresses and unauthorized outbound TCP 8443 traffic must be blocked, svc-backup must be disabled, and the compromised SSH key must be revoked across the environment. Server-to-server SSH must be temporarily restricted to approved administrative workstations and managed jump hosts.

Eradication

Both compromised hosts must be rebuilt because the attacker obtained root access. Apache must be upgraded to version 2.4.51 or later, and policykit/pkexec must be updated to a vendor-supported version that fixes the identified privilege-escalation vulnerability. The webshell, both beacon processes, the malicious cron entry, the System Log Forwarder service, the UID 0 svc-backup accounts, the attacker-controlled SSH key, and all staging files must be preserved as evidence and removed from the rebuilt environment.

The harvested SSH private key must be revoked and replaced everywhere its public key was trusted. The svc-backup password and associated keys must be invalidated across the fleet. Administrative credentials, application secrets, database credentials, API tokens, and other private keys accessible to root on either compromised server must be

identified and rotated.

Eradication must be verified through authenticated vulnerability scanning, Apache configuration review, file-integrity comparison, account and UID review, inspection of SSH keys, cron and systemd enumeration, endpoint scans, and network-telemetry review. The four new detection rules must be run against the rebuilt hosts and historical evidence.

Recovery

MTB-FILE01 should return first in a restricted internal network so its services can be tested under monitoring.

MTB-WEB01 should return last because it is internet-facing and was the original entry point.

Only a backup created before 28 August 2026 at 01:52:17 UTC should be trusted, preferably a verified backup from 27 August or earlier. Before restoration, the backup must be mounted offline, hash-verified, scanned for known indicators, compared with a trusted baseline, and reviewed for unauthorized accounts, keys, cron entries, systemd services, packages, and configuration changes.

Before MTB-WEB01 returns to the internet, Apache and policykit must be patched, unnecessary CGI functionality must be disabled, the filesystem root must use Require all denied, and access must be granted only to required web directories. Compromised credentials must be rotated, the web root must match a known-good version, the new detection rules must be active, and testing must confirm that traversal and privilege escalation no longer work.

The SOC must conduct continuous heightened monitoring for at least 30 days after restoration. Monitoring must cover the four new detection rules, unauthorized UID 0 accounts, SSH-key changes, long-lived external connections, non-allowlisted destinations, and unusually large outbound transfers.

Root Cause and Contributing Factors

The technical root cause was an unpatched Apache 2.4.49 path-traversal and remote-command-execution vulnerability, CVE-2021-41773, combined with an exposed CGI configuration and a permissive Require all granted directive.

CVE-2021-4034 in policykit then allowed the attacker to elevate from the web-server account to root.

The broader patch-management process did not connect threat intelligence, internet exposure, asset criticality, ownership, remediation deadlines, and verification. The vulnerability had been publicly known and actively exploited for years, but the process did not force timely remediation or escalate the overdue exposure.

The vulnerability assessment ran after the first successful exploitation because the scan cadence was not appropriate for an internet-facing banking system. Weekly authenticated scans, daily external-exposure monitoring, and event-driven scanning for newly exploited vulnerabilities would have reduced the window, although this requires additional scanner capacity, analyst validation time, and maintenance coordination.

A reusable SSH private key was stored on an internet-facing DMZ server and trusted by an internal host. This violated least privilege and allowed the compromise to cross network boundaries. DMZ systems should not initiate direct administrative connections to internal servers without an approved requirement, restricted firewall path, and managed jump host.

The perimeter firewall recorded the five-minute beaconing, but no detection correlated its regularity, non-allowlisted destination, process ownership, and the criticality of the source host. The control gap was not missing data; it was missing analytics, alerting, and timely human review.

Recommendations

1. Infrastructure Engineering must rebuild MTB-WEB01 and MTB-FILE01 from verified pre-28 August sources and complete vulnerability validation before either host returns to production. Due: 5 September 2026.
2. The Linux Platform Team must upgrade Apache to version 2.4.51 or later, install the vendor-supported policykit fix, disable unnecessary CGI functionality, and enforce Require all denied at the filesystem root. Due: before MTB-WEB01 is reconnected to the internet.
3. Identity and Access Management must disable svc-backup, revoke the compromised SSH key, identify every host that trusted it, rotate exposed credentials and secrets, and document the new key inventory. Due: 5 September 2026.
4. Network Security must block known attacker infrastructure, restrict DMZ-to-server SSH, require approved administrative access through a managed jump host, and limit outbound server traffic to documented destinations and ports. Due: 11 September 2026.

5. The SOC Detection Engineering team must deploy and test all four Sigma rules and add monitoring for unauthorized UID 0 accounts, SSH-key changes, PwnKit exploitation, local data staging, and large outbound transfers. Due: 7 September 2026.
6. Vulnerability Management must implement weekly authenticated scanning for internet-facing systems, daily external-exposure monitoring, and event-driven scans when high-risk vulnerabilities enter the KEV catalog. Due: 11 September 2026.
7. Security Operations must monitor both restored hosts continuously for 30 days and document all alerts, investigations, and tuning changes. Owner: SOC Manager. Start: immediately upon restoration.
8. Governance, Risk and Compliance must establish remediation deadlines, named owners, exception approval, escalation, and closure verification for vulnerabilities affecting internet-facing or critical assets. Due: 30 September 2026.

14.1- Reflection Day 4

1. Your Day 3 forensic report gave a start date that turned out to be wrong by several days. The investigation was not careless. Explain what made the disk evidence misleading, and what that tells you about relying on any single evidence source.

The disk evidence was misleading because MTB-WEB01's older Apache logs had already rotated, while the perimeter firewall records were stored somewhere outside the affected server. The disk image accurately showed what remained on the host, but it could not show evidence that had been deleted or never existed there. This demonstrates why an investigation must correlate host, network, authentication, and centralized logging sources instead of treating one source as the complete timeline.

2. The attacker used a different address, filename and persistence mechanism on the second host. What does that tell you about how they operate, and what would you now assume about a third host if you found one?

The different address, filename, and persistence mechanism show that attackers can change visible indicators without changing their overall behavior. They reused the same basic goals—persistence, command and control, lateral movement, and data theft—while changing how those activities looked. If a third compromised host were found, I would assume its specific indicators might be different and hunt for the same behaviors rather than only searching for known names and addresses.

3. MTB-DVWA01 showed the same beacon-like periodicity that got MTB-WEB01 flagged, and it was benign. Explain what actually distinguished them, and why an analyst who cannot make that distinction is a problem for their team.

MTB-DVWA01's connections were generated by an approved monitoring agent communicating with its documented allowlisted endpoint every 300 seconds. MTB-WEB01's connections came from an unexplained root-owned process, used an unauthorized destination, and were tied to malicious persistence and other compromise evidence. An analyst who cannot use asset context to distinguish these situations will either waste time investigating normal activity or ignore genuinely dangerous behavior as another routine connection.

4. Why is documenting what you checked on the clean hosts as important as documenting what you found on the compromised ones?

Documenting the clean-host checks shows that the conclusion was based on evidence rather than an empty search result. It tells the reader which accounts, processes, addresses, persistence mechanisms, authentication logs, and network records were reviewed. It also exposes the investigation's limits because a clean result against known indicators does not guarantee that an attacker using different techniques would have been found.

14.2 - Reflection Day 5

1. Explain the Pyramid of Pain in your own words, using an example from this incident, and say why blocking the C2 address after Day 3 achieved so little.

The Pyramid of Pain ranks indicators according to how difficult they are for an attacker to replace. Blocking 185.220.101.47 achieved little because the attacker was already using 185.220.102.244 on MTB-FILE01, and switching addresses required almost no change to the operation. Detecting the repeated behavior of a persistent root-owned process maintaining an outbound connection is more valuable because evading that detection requires the attacker to redesign how the attack works.

2. The account-creation rule in Section 10.3 detects the attacker using a warning the operating system produced by itself. Why is a detection built on the target's own logging harder to evade than one built on the attacker's tooling?

The UID 0 warning is produced by the operating system whenever that unusual account condition occurs, regardless of the attacker's chosen username or tools. The attacker can rename a program or account easily, but the same system behavior still generates the warning. Evading it would require abandoning the technique or interfering with trusted logging, which is more difficult and may create additional detectable activity.

3. Every one of your rules will produce false positives eventually. Explain why the falsepositives field matters more to the analyst receiving the alert than to the person writing the rule.

The false-positive field gives the receiving analyst immediate context about legitimate activity that could produce the alert. Without it, the analyst must determine from scratch whether an administrator, deployment process, vulnerability scan, or approved service caused the event. Clear false-positive guidance improves triage speed, reduces alert fatigue, and helps prevent a real attack from being dismissed after repeated benign alerts.

4. The egress log recorded all seven and a half hours of beaconing. The data existed and nobody acted on it. What does that suggest about where security programmes actually fail, and how would you fix it here?

The beaconing shows that collecting logs does not provide protection unless someone converts the data into useful detection logic and responds to the resulting alerts. Security programs often fail at correlation, tuning, ownership, and escalation rather than at data collection itself. Meridian Trust should implement behavioral detection for repeated non-allowlisted connections, connect those alerts to asset and process context, and assign the SOC clear response deadlines.